Start here: itsdangerous

itsdangerous signs data so you can hand it to an untrusted party (a browser cookie, an email link, a URL) and later prove it came back unchanged. It signs; it does not encrypt. Anyone holding a token can base64-decode it and read the payload.

```
pip install itsdangerous
```

```
from itsdangerous import URLSafeSerializer

auth_s = URLSafeSerializer("secret key", "auth")   # (secret_key, salt)
token = auth_s.dumps({"id": 5, "name": "itsdangerous"})
print(token)
# eyJpZCI6NSwibmFtZSI6Iml0c2Rhbmdlcm91cyJ9.6YP6T0BaO67XP--9UzTrmurXSmg
print(auth_s.loads(token)["name"])
# itsdangerous
```

`loads` returns the data or raises `BadSignature`. That raise is the whole security check, so never swallow it.

This page covers the checkout at `2.3.0.dev` (Python 3.10+, no runtime dependencies). The examples above and below were run against it, and the test suite passed (297 tests). The `uv`/`tox` commands were read from CI config, not run.

Pick a class

Most code wants a serializer, not a raw signer. A serializer turns Python data into JSON, then signs it.

Class	Input	Output	Adds
`URLSafeSerializer`	any JSON value	`str` safe for URLs	compact JSON, zlib, base64
`URLSafeTimedSerializer`	any JSON value	`str` safe for URLs	the above plus a timestamp, so `loads(max_age=...)` works
`Serializer`	any JSON value	`str`, raw JSON + signature	nothing; payload has spaces and quotes
`TimedSerializer`	any JSON value	`str`, raw JSON	timestamp
`Signer`	`str`/`bytes`	`bytes`	signature only
`TimestampSigner`	`str`/`bytes`	`bytes`	timestamp

For password reset or email confirmation links, use `URLSafeTimedSerializer`:

```
from itsdangerous import URLSafeTimedSerializer, SignatureExpired, BadSignature

s = URLSafeTimedSerializer(SECRET_KEY, salt="password-reset")
token = s.dumps("user@example.com")

try:
    email = s.loads(token, max_age=3600)   # seconds
except SignatureExpired:
    ...  # valid signature, too old; e.date_signed says when it was made
except BadSignature:
    ...  # tampered, wrong key, or wrong salt
```

Map of the code

All source is in `src/itsdangerous/` (about 1,200 lines); `__init__.py` re-exports the public API.

File	What lives there
`signer.py`	`Signer`, key derivation, `HMACAlgorithm`, `NoneAlgorithm`
`timed.py`	`TimestampSigner`, `TimedSerializer`
`serializer.py`	`Serializer`: payload dump/load, signer construction, fallback signers
`url_safe.py`	`URLSafeSerializerMixin`: compact JSON, zlib, base64
`encoding.py`	URL-safe base64 without padding, int/bytes helpers
`exc.py`	the exception hierarchy
`tests/test_itsdangerous/`	one test module per source module
`docs/`	Sphinx docs; `concepts.rst` is the best background read

How a token is built

`URLSafeTimedSerializer.dumps(obj)` runs through three layers:

1. `dump_payload` (`url_safe.py:55`) writes compact JSON. If zlib makes it shorter, it compresses and prefixes the payload with `.`. Then it base64-encodes.
2. `make_signer` (`serializer.py:278`) builds a fresh signer from `secret_keys`, `salt`, and `signer_kwargs`.
3. `TimestampSigner.sign` (`timed.py:45`) appends `.timestamp`, then `.signature`.

The result is `payload.timestamp.signature`. The signature is HMAC-SHA1 over everything before it, keyed by a key derived from `salt + b"signer" + secret_key` (the `django-concat` derivation, `signer.py:202`).

`loads` reverses this. It tries the main signer, then each fallback signer (`iter_unsigners`, `serializer.py:287`). Each signer tries every secret key, newest first. An expired timestamp stops the search; it does not fall through to the next signer (`timed.py:213`).

Configuration you will touch

Setting	Default	Notes
`secret_key`	required	A key, or a list ordered oldest to newest. Signing uses the last one; verifying tries all.
`salt`	`b"itsdangerous"`	Not secret. Use a different salt per purpose so a token from one feature can't be replayed in another.
`signer_kwargs={"digest_method": ...}`	`hashlib.sha1`	SHA-1 is fine inside HMAC (`docs/concepts.rst`). SHA-512 makes tokens longer.
`fallback_signers`	`[]`	Signer kwargs to try after the main one fails. Use it to migrate old tokens.
`Signer(sep=...)`	`b"."`	Must not be a base64 character (letters, digits, `-_=`), or the constructor raises `ValueError`.

Generate a key once and load it from the environment, not source control:

```
python3 -c 'import os; print(os.urandom(16).hex())'
```

To move from SHA-1 to SHA-512 without breaking live tokens, sign new tokens with the new digest and accept the old ones:

```
import hashlib
s = URLSafeSerializer(
    SECRET_KEY,
    signer_kwargs={"digest_method": hashlib.sha512},
    fallback_signers=[{"digest_method": hashlib.sha1}],
)
```

Errors

Catch `BadData` to catch everything the library raises.

Exception	Parent	Raised when
`BadSignature`	`BadData`	Signature missing or wrong. `e.payload` holds the unverified bytes.
`BadTimeSignature`	`BadSignature`	A timed token has a bad signature or a malformed or missing timestamp
`SignatureExpired`	`BadTimeSignature`	Older than `max_age`, or dated in the future. Has `e.date_signed` (UTC-aware).
`BadPayload`	`BadData`	Signature valid but the payload won't decode. Not a `BadSignature` subclass.

Gotchas

- `TimedSerializer.loads(token, max_age=None, return_timestamp=False, salt=None)` takes `max_age` second. Calling `s.loads(token, "my-salt")` passes the salt as `max_age` and fails with `TypeError: '>' not supported between instances of 'int' and 'str'`. Always pass keywords.
- Without `max_age`, a timed token never expires. The timestamp is recorded but only checked when you ask.
- Tuples come back as lists, and sets fail with `TypeError` at `dumps`, because the default serializer is JSON.
- `Signer` defaults to salt `b"itsdangerous.Signer"` but `Serializer` defaults to `b"itsdangerous"`. A raw `Signer` and a `Serializer` with the same key produce different signatures.
- `loads_unsafe` returns `(valid, payload)` without trusting the signature. Debugging only, and never with a pickle serializer.
- Changing the secret key or salt invalidates every outstanding token. Rotate by appending to the key list instead.

Day 1: run the tests

The project uses `uv` and `tox` (see `.devcontainer/on-create-command.sh` and `.github/workflows/tests.yaml`):

```
uv sync                          # creates .venv with dev, tests, typing groups
uv run tox run -e py3.13         # tests on one Python
uv run tox run -e style          # ruff + pre-commit hooks
uv run tox run -e typing         # mypy (strict) and pyright
uv run tox run -e docs           # sphinx build, warnings are errors
```

Without `uv`, a plain venv works:

```
python3 -m venv .venv
.venv/bin/pip install -e . pytest freezegun
.venv/bin/pytest -q
```

Pytest treats every warning as an error (`filterwarnings = ["error"]` in `pyproject.toml`), so a new `DeprecationWarning` fails the suite. Per `.github/pull_request_template.md`, open an issue before a PR, and ship a failing-then-passing test plus a `CHANGES.rst` entry.

Next: itsdangerous.palletsprojects.com for the full API reference.